16.07 Dynamics Problem Set 6

Due Wednesday, November 18

Spacecraft Dynamics With Reaction Control Thrusters

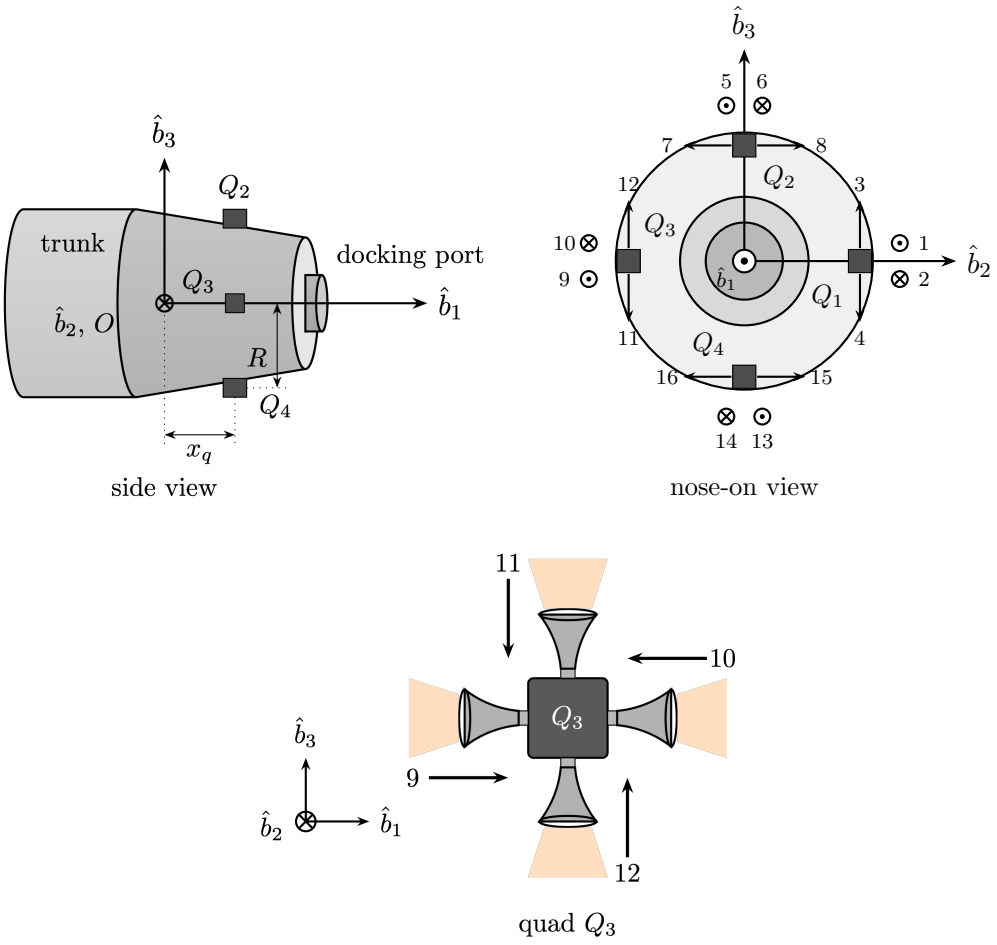


Figure 1: Simplified Dragon-like spacecraft with 16 RCS thrusters arranged in four quads $Q_1, \ldots, Q_4$. The body frame B has its origin at the vehicle center of mass O, with $\hat{b}_1$ along the symmetry axis toward the docking port. Top left: side view ($\hat{b}_2$ into the page; Q_1 is hidden behind the vehicle). Top right: view looking at the nose ($\hat{b}_1$ out of the page). Bottom: schematic of quad Q_3 in the same orientation as the side view, showing its four engine bells; each thruster's exhaust leaves through its bell, and the arrow next to it shows the resulting force on the vehicle. Arrows and the $\odot/\otimes$ symbols show the direction of the force each thruster applies *to the vehicle* (opposite to its exhaust), labeled with the thruster number. Drawings are not to scale.

In this problem set, you'll build a full six-degree-of-freedom simulation of a spacecraft in low-Earth orbit, loosely modeled on SpaceX's Dragon. Dragon maneuvers using 16 Draco thrusters, grouped into four quads of four thrusters each. Each Draco produces about 400 N of thrust in vacuum. You'll derive how the thruster commands map into a force and torque on the vehicle, write the Newton–Euler equations of motion in manipulator form, compare two fixed-step integrators, and pose thruster allocation as an optimization problem.

We'll use the following notation and assumptions throughout:

- N is an Earth-centered inertial (ECI) frame. B is the body frame shown in Fig. 1, with origin at the vehicle's center of mass O and axes along its principal axes. As in Problem Sets 4 and 5, a left superscript denotes the frame in which a vector's components are expressed.
- Attitude is represented by a unit quaternion $\mathbf{a}$, using the conventions from class and Problem Set 5, so the rotation matrix $\mathbf{A} = \mathbf{A}(\mathbf{a})$. *Note the change of notation:* in Problem Sets 4 and 5 we called the rotation matrix $\mathbf{Q}$ and the quaternion $\mathbf{q}$. Here we'll call them $\mathbf{A}$ and $\mathbf{a}$ (for "attitude"), so that we can use $\mathbf{q}$ for the configuration vector in Part 2.
- ${}^N\mathbf{r}$ and ${}^N\dot{\mathbf{r}}$ are the position and velocity of the vehicle's center of mass relative to the center of the Earth, and ${}^B\boldsymbol{\omega}$ is the vehicle's angular velocity relative to N, expressed in the body frame.
- The Earth is assumed to be perfectly spherical with gravitational parameter $\mu = 3.986004418 \times 10^{14}$ m^3/s^2. There are no other external forces or torques.
- The vehicle is rigid and its mass and inertia are constant (we neglect the propellant consumed).
- Each thruster i produces a force of magnitude u_i along a fixed direction in the body, and can be throttled continuously over $0 \le u_i \le u_{\max}$. (Real Dracos are pulsed on and off, but a continuous throttle is a good model of their average effect.)

We'll work in SI units throughout.

Part 1: Vehicle Model and the Input Jacobian

The vehicle's mass properties and thruster specifications are given in Table 1. The mass is close to Crew Dragon's in-orbit mass with its trunk attached. The inertia matrix is an illustrative estimate, since SpaceX doesn't publish one.

Each quad is a pod of four thrusters, and we'll treat all four thrusters in a pod as acting at the pod's location. The pods all sit $x_q = 1.5$ m forward of O, at a distance $R = 1.8$ m from the $\hat{b}_1$ axis (Fig. 1). Quad Q_k contains thrusters $4k-3$ through $4k$, numbered as in Fig. 1. The positions ${}^B\mathbf{r}_i$ of the thrusters relative to O, in body-frame components, are

$$
\begin{aligned}
{}^B\mathbf{r}_1 = {}^B\mathbf{r}_2 = {}^B\mathbf{r}_3 = {}^B\mathbf{r}_4 &= \begin{bmatrix} 1.5 & 1.8 & 0 \end{bmatrix}^T \text{ m} && (Q_1),\\
{}^B\mathbf{r}_5 = {}^B\mathbf{r}_6 = {}^B\mathbf{r}_7 = {}^B\mathbf{r}_8 &= \begin{bmatrix} 1.5 & 0 & 1.8 \end{bmatrix}^T \text{ m} && (Q_2),\\
{}^B\mathbf{r}_9 = {}^B\mathbf{r}_{10} = {}^B\mathbf{r}_{11} = {}^B\mathbf{r}_{12} &= \begin{bmatrix} 1.5 & -1.8 & 0 \end{bmatrix}^T \text{ m} && (Q_3),\\
{}^B\mathbf{r}_{13} = {}^B\mathbf{r}_{14} = {}^B\mathbf{r}_{15} = {}^B\mathbf{r}_{16} &= \begin{bmatrix} 1.5 & 0 & -1.8 \end{bmatrix}^T \text{ m} && (Q_4).
\end{aligned}
$$

Quantity	Symbol	Value
Mass	m	12,500 kg
Inertia matrix about O, in B	$\mathbf{J}$	$\mathrm{diag}\,(22,\ 48,\ 52) \times 10^3$ kg m^2
Number of thrusters		16 (4 quads $\times$ 4)
Maximum thrust per thruster	$u_{\max}$	400 N

Table 1: Vehicle parameters.

(a) **Thrust directions:** Using Fig. 1, find the unit vector ${}^B\mathbf{d}_i$ along which each thruster's force vector points in the body frame. Write these in a table.

(b) **Input Jacobian:** Write the force and torque that thruster i applies to the vehicle, in terms of u_i, ${}^B\mathbf{r}_i$, and ${}^B\mathbf{d}_i$. The total *wrench* (force and torque) on the vehicle in terms of the thrust commands $\mathbf{u} = [u_1\ \cdots\ u_{16}]^T$ can be written in matrix form,

$$ {}^B\mathbf{w} = \begin{bmatrix} {}^B\mathbf{F} \\ {}^B\boldsymbol{\tau} \end{bmatrix} = \mathbf{B}\,\mathbf{u}, $$

where the 6×16 matrix $\mathbf{B}$ is called the *input Jacobian*. Write $\mathbf{B}$ in terms of the ${}^B\mathbf{r}_i$ and ${}^B\mathbf{d}_i$. Why is $\mathbf{B}$ constant in the body frame?

Part 2: Equations of Motion

We'll write the translational dynamics in inertial (ECI) N-frame components, rather than in the body frame. Gravity takes its simplest form in N, but the thrust force, which is naturally expressed in B, must be rotated into N using the attitude. The rotational dynamics, as in Problem Set 4, are written in body-frame components, where $\mathbf{J}$ is constant.

(a) **Newton–Euler equations:** Write Newton's 2nd law for the translational motion of the center of mass in inertial N-frame components, including both gravity and the thrust force ${}^B\mathbf{F}$. Remember that the thrust force from Part 1 is expressed in B, so you'll have to rotate it. Then write Euler's equation for the rotational motion in B-frame components, using the thrust torque ${}^B\boldsymbol{\tau}$ from Part 1.

(b) **Manipulator form:** Define the configuration and velocity vectors

$$ \mathbf{q} = \begin{bmatrix} {}^N\mathbf{r} \\ \mathbf{a} \end{bmatrix} \in \mathbb{R}^7, \qquad \mathbf{v} = \begin{bmatrix} {}^N\dot{\mathbf{r}} \\ {}^B\boldsymbol{\omega} \end{bmatrix} \in \mathbb{R}^6. $$

Using the quaternion kinematics from class and Problem Set 5 (with $\mathbf{q}$ there renamed $\mathbf{a}$), write the kinematics in the form $\dot{\mathbf{q}} = \mathbf{N}(\mathbf{q})\,\mathbf{v}$, and give the 7×6 matrix $\mathbf{N}(\mathbf{q})$. Then write the dynamics in *manipulator form*,

$$ \mathbf{M}(\mathbf{q})\,\dot{\mathbf{v}} + \mathbf{c}(\mathbf{q}, \mathbf{v}) + \mathbf{g}(\mathbf{q}) = \mathbf{B}(\mathbf{q})\,\mathbf{u}, $$

where $\mathbf{c}(\mathbf{q}, \mathbf{v}) \in \mathbb{R}^6$ collects all of the velocity-dependent terms and $\mathbf{g}(\mathbf{q}) \in \mathbb{R}^6$ collects the gravity terms. Write out $\mathbf{M}$, $\mathbf{c}$, $\mathbf{g}$, and the 6×16 matrix $\mathbf{B}(\mathbf{q})$ in terms of the input Jacobian $\mathbf{B}$ from Part 1. Why is $\mathbf{B}$ constant, while $\mathbf{B}(\mathbf{q})$ is not?

Part 3: Simulation

So far, we've used adaptive integrators like `ode45` and `solve_ivp`. A flight computer, however, usually runs at a fixed rate: it reads its sensors, computes thruster commands, and holds those commands constant until the next update. In this part, you'll simulate the vehicle with two simple fixed-step integrators running at 20 Hz, i.e., with time step $h = 0.05$ s. Writing $\mathbf{x}_k = \mathbf{x}(kh)$ and holding $\mathbf{u}_k$ constant over each step, the two schemes are:

$$\begin{aligned} \text{Forward Euler:} \quad & \mathbf{x}_{k+1} = \mathbf{x}_k + h\,\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k), \\ \text{Explicit midpoint:} \quad & \mathbf{x}_{k+1} = \mathbf{x}_k + h\,\mathbf{f}\big(\mathbf{x}_k + \tfrac{h}{2}\,\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k),\ \mathbf{u}_k\big)\,. \end{aligned}$$

Forward Euler is first-order accurate, and explicit midpoint (a 2nd-order Runge–Kutta method) is second-order accurate.

We'll start the vehicle in a circular orbit similar to that of the International Space Station (ISS), at an altitude of about 420 km and an inclination of $i = 51.64°$. The initial conditions are

$${}^N\mathbf{r}(0) = \begin{bmatrix} r_0 \\ 0 \\ 0 \end{bmatrix}, \quad {}^N\dot{\mathbf{r}}(0) = v_c \begin{bmatrix} 0 \\ \cos i \\ \sin i \end{bmatrix}, \quad \mathbf{a}(0) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad {}^B\boldsymbol{\omega}(0) = \begin{bmatrix} 1 \\ 0.5 \\ -0.5 \end{bmatrix} \text{ deg/s},$$

where

$$r_0 = 6{,}797{,}956.058 \text{ m}, \qquad v_c = \sqrt{\mu/r_0} = 7{,}657.371391 \text{ m/s}, \qquad {}^N\dot{\mathbf{r}}(0) = \begin{bmatrix} 0 \\ 4{,}752.168573 \\ 6{,}004.350962 \end{bmatrix} \text{ m/s}.$$

All thrusters are off, $\mathbf{u} = \mathbf{0}$. This orbit's period is exactly $P = 2\pi\sqrt{r_0^3/\mu} = 5{,}578.0$ s (about 93 minutes), so after one period the vehicle should return exactly to its starting position. Compute v_c in your code from μ and r_0, rather than typing in the rounded value.

(a) **First-order ODE:** Define the 13-dimensional state vector $\mathbf{x} = [\mathbf{q}^T \ \ \mathbf{v}^T]^T$ and write the full dynamics as a first-order vector ODE $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$. Implement $\mathbf{f}$ as a function in MATLAB or Python, reusing your quaternion functions from Problem Set 5.

(b) **Integrators:** Implement one step of each scheme as a function that takes $\mathbf{x}_k$, $\mathbf{u}_k$, and h and returns $\mathbf{x}_{k+1}$. At the end of every step, renormalize the quaternion, $\mathbf{a} \leftarrow \mathbf{a}/\|\mathbf{a}\|$, before returning $\mathbf{x}_{k+1}$.

(c) **One orbit:** Using each scheme with $h = 0.05$ s, simulate exactly one orbital period, $K = P/h = 111{,}560$ steps. For each scheme, report:

- The orbital position error after one period, $\|{}^N\mathbf{r}(P) - {}^N\mathbf{r}(0)\|$.
- The relative error in the angular momentum after one period, $\|{}^N\mathbf{h}(P) - {}^N\mathbf{h}(0)\| / \|{}^N\mathbf{h}(0)\|$, where ${}^N\mathbf{h} = \mathbf{A}(\mathbf{a})\,\mathbf{J}\,{}^B\boldsymbol{\omega}$.

(d) **Comparison:**

- Which scheme is more accurate, and by roughly how much?

- Explicit midpoint evaluates $\mathbf{f}$ twice per step, so it costs about twice as much as forward Euler. To compare at equal cost, rerun forward Euler with $h = 0.025$ s (40 Hz, $K =$ 223,120 steps) for one orbit. How does its position error compare with your 20 Hz midpoint result? Explain using the order of accuracy of each method.
- A docking port is about a meter across. Which of these integrators would you trust to simulate a rendezvous with the ISS?

Part 4: Thruster Allocation

A spacecraft's guidance and control software typically decides what force and torque it wants, ${}^{B}\mathbf{w}_{\text{des}}$, and then has to choose thruster commands $\mathbf{u}$ that produce it. This is called *thruster allocation*. Since $\mathbf{B}$ has many more columns than rows, there are infinitely many solutions to $\mathbf{B}\mathbf{u} = {}^{B}\mathbf{w}_{\text{des}}$, and we need a principled way to choose one.

(a) **Naive pseudoinverse:** Suppose we want a pure $+\hat{b}_2$ translation force of 200 N with no torque:

$$ {}^{B}\mathbf{w}_{\text{des}} = \begin{bmatrix} 0 & 200 & 0 & 0 & 0 & 0 \end{bmatrix}^{T}. $$

Since $\mathbf{B}$ is not square, compute $\mathbf{u} = \mathbf{B}^{+}\,{}^{B}\mathbf{w}_{\text{des}}$, where $\mathbf{B}^{+}$ is the Moore–Penrose pseudoinverse (`pinv` in MATLAB or `numpy.linalg.pinv`). Verify that $\mathbf{B}\mathbf{u} = {}^{B}\mathbf{w}_{\text{des}}$. Can the thrusters actually produce this $\mathbf{u}$? Why or why not? *Hint: Can thrusters produce negative thrust?*

(b) **Thruster allocation as a QP:** We can fix this by posing allocation as a constrained optimization problem. Among all thrust commands that produce the desired wrench and respect the thruster limits, we'll choose the one with the least total "effort," measured by $\|\mathbf{u}\|^2$:

$$ \begin{aligned} \underset{\mathbf{u}\in\mathbb{R}^{16}}{\text{minimize}} \quad & \tfrac{1}{2}\,\mathbf{u}^{T}\mathbf{u} \\ \text{subject to} \quad & \mathbf{B}\,\mathbf{u} = {}^{B}\mathbf{w}_{\text{des}}, \\ & 0 \le u_i \le u_{\max}, \quad i = 1, \ldots, 16. \end{aligned} $$

This is a *quadratic program* (QP): a quadratic objective with linear equality and inequality constraints. Because the objective is strictly convex and the constraints are linear, a feasible QP has a unique global minimum that can be found reliably and quickly.

Solving a QP like this takes only a few lines of code with a convex optimization package. In MATLAB, using CVX (`https://github.com/cvxr/CVX`):

```
cvx_begin quiet
    variable u(16)
    minimize( 0.5 * sum_square(u) )
    subject to
        B * u == w_des;
        0 <= u <= u_max;
cvx_end
```

In Python, using CVXPY (`pip install cvxpy`):

```
import cvxpy as cp

u = cp.Variable(16)
```

```
objective = cp.Minimize(0.5 * cp.sum_squares(u))
constraints = [B @ u == w_des, u >= 0, u <= u_max]
prob = cp.Problem(objective, constraints)
prob.solve()
print(prob.status, u.value)
```

Use this code to solve the QP for two desired wrenches:

- The pure $+\hat{b}_2$ force of 200 N from part (a).
- A combined 300 N force along $+\hat{b}_1$ and a 100 N m roll torque about $+\hat{b}_1$.

For each, report $\mathbf{u}$, verify that $\mathbf{Bu} = {}^{B}\mathbf{w}_{\text{des}}$ and that all constraints are satisfied, and explain physically which thrusters fire and why. For the first wrench, how is the QP solution related to the pseudoinverse solution from part (a)? *Hint: Look at each opposed pair of thrusters.*